Nagar Yuwak Shikshan Sanstha's
Yeshwantrao Chavan College of Engineering
(Empowered Autonomous Institution under R.T.M Nagpur University)
Hingna Road, Wanadongri, Nagpur - 441 110
NAAC A++ Grade
Ph.: 07104-237919, 234623, 329249, 329250 Fax: 07104-232376, Website: www.ycce.edu
Department of CSE-IoT

Vision of the Department

To be a preferred hub for nurturing computer engineering aspirants through innovative, ethical, value-based technical education, impactful research, and industry collaboration, to become successful professionals contributing for the betterment of society.

Mission of the Department

To nurture computer engineering professionals by offering: Dynamic learning environment that encourages problem-solving using emerging technology, · National Education Policy aligned evidence-based technical education, fostering innovation, research, and industry–institute collaboration, Holistic, ethical and sustainable development for lifelong success.

Session 2025-2026(EVEN)

Vision: Dream of where you want.	**Mission:** Means to achieve Vision

Program Educational Objectives of the program (PEO): (broad statements that describe the professional and career accomplishments)

PEO1	**Preparation**	**P: Preparation**	**Pep-CL abbreviation pronounce as Pep-si-lL easy to recall**
PEO2	**Core Competence**	**E: Environment (Learning Environment)**	
PEO3	**Breadth**	**P: Professionalism**	
PEO4	**Professionalism**	**C: Core Competence**	
PEO5	**Learning Environment**	**L: Breadth (Learning in diverse areas)**	

Program Outcomes (PO): (statements that describe what a student should be able to do and know by the end of a program)

Keywords of POs:
Engineering knowledge, Problem analysis, Design/development of solutions, Conduct Investigations of Complex Problems, Engineering Tool Usage, The Engineer and The World, Ethics, Individual and Collaborative Team work, Communication, Project Management and Finance, Life-Long Learning

PSO Keywords: Cutting edge technologies, Research

"I am an engineer, and I know how to apply engineering knowledge to investigate, analyse and design solutions to complex problems using tools for entire world following all ethics in a collaborative way with proper management skills throughout my life." *to contribute to the development of cutting-edge technologies and Research*.

Integrity: I will adhere to the Laboratory Code of Conduct and ethics in its entirety.

Sheikh Wasimuddin
Name and Signature of Student and Date
(Signature and Date in Handwritten)

Nagar Yuwak Shikshan Sanstha's
Yeshwantrao Chavan College of Engineering
(Empowered Autonomous Institution under R.T.M Nagpur University)
Hingna Road, Wanadongri, Nagpur - 441 110
NAAC A++ Grade
Ph.: 07104-237919, 234623, 329249, 329250 Fax: 07104-232376, Website: www.ycce.edu
Department of CSE-IoT

Vision of the Department

To be a preferred hub for nurturing computer engineering aspirants through innovative, ethical, value-based technical education, impactful research, and industry collaboration, to become successful professionals contributing for the betterment of society.

Mission of the Department

To nurture computer engineering professionals by offering: Dynamic learning environment that encourages problem-solving using emerging technology, · National Education Policy aligned evidence-based technical education, fostering innovation, research, and industry–institute collaboration, Holistic, ethical and sustainable development for lifelong success.

Session	2024-25 (EVEN)	Course Name	Competitive Coding
Semester	6	Course Code	23IOT1605
Roll No	63	Name of Student	Sheikh Wasimuddin
Practical Number	Competitive Coding DS Slot - 1		

Advanced Data Structure Problems in C

1. **Design and implement a Least Recently Used (LRU) Cache with capacity C.**

Answer)

```
// sheikh wasimuddin 63 CSEIOT_CC LAB DS
#include <stdio.h>
#include <stdlib.h>
struct Node
{
   int key, value;
   struct Node *prev, *next;
};
struct Node *head = NULL, *tail = NULL;
struct Node *cache[1000];
int capacity, size = 0;
struct Node* createNode(int key, int value)
{
   struct Node* node = (struct Node*)malloc(sizeof(struct Node));
   node->key = key;
   node->value = value;
   node->prev = node->next = NULL;
   return node;
}
void moveToFront(struct Node* node)
{
   if (node == head)
      return;
   if (node->prev)
      node->prev->next = node->next;
   if (node->next)
      node->next->prev = node->prev;
   if (node == tail)
      tail = node->prev;
   node->prev = NULL;
   node->next = head;
```

```
    if (head)
        head->prev = node;
    head = node;
    if (!tail)
        tail = node;
}
void insertFront(struct Node* node)
{
    node->next = head;
    node->prev = NULL;
    if (head)
        head->prev = node;
    head = node;
    if (!tail)
        tail = node;
}
int get(int key)
{
    if (cache[key] == NULL)
        return -1;
    moveToFront(cache[key]);
    return cache[key]->value;
}
void put(int key, int value)
{
    if (cache[key] != NULL)
    {
        cache[key]->value = value;
        moveToFront(cache[key]);
        return;
    }
    struct Node* node = createNode(key, value);
    if (size == capacity)
    {
        cache[tail->key] = NULL;
        struct Node* temp = tail;
        tail = tail->prev;
        if (tail)
            tail->next = NULL;
        free(temp);
        size--;
    }
    insertFront(node);
    cache[key] = node;
    size++;
}
```

```
int main()
{
    int n, type, key, value;
    printf("Enter capacity: ");
    scanf("%d", &capacity);
    for (int i = 0; i < 1000; i++)
        cache[i] = NULL;
    printf("Enter number of operations: ");
    scanf("%d", &n);
    while (n--)
    {
        printf("Enter type (1-put, 2-get): ");
        scanf("%d", &type);
        if (type == 1)
        {
            printf("Enter key and value: ");
            scanf("%d %d", &key, &value);
            put(key, value);
        }
        else
        {
            printf("Enter key: ");
            scanf("%d", &key);
            int res = get(key);
            if (res == -1)
                printf("Not found\n");
            else
                printf("Value: %d\n", res);
        }
    }
    return 0;
}
```

Vision of the Department

To be a preferred hub for nurturing computer engineering aspirants through innovative, ethical, value-based technical education, impactful research, and industry collaboration, to become successful professionals contributing for the betterment of society.

Mission of the Department

To nurture computer engineering professionals by offering: Dynamic learning environment that encourages problem-solving using emerging technology, · National Education Policy aligned evidence-based technical education, fostering innovation, research, and industry–institute collaboration, Holistic, ethical and sustainable development for lifelong success.

main.c

```c
// sheikh wasimuddin 63 CSEIOT_CC LAB DS
#include <stdio.h>
#include <stdlib.h>
struct Node
{
int key, value;
struct Node *prev, *next;
};
struct Node *head = NULL, *tail = NULL; struct Node *cache[1000];
```

input

```
Enter capacity: 2
Enter number of operations: 5
Enter type (1-put, 2-get): 1
Enter key and value: 1 10
Enter type (1-put, 2-get): 1
Enter key and value: 2 20
Enter type (1-put, 2-get): 2
Enter key: 1
Value: 10
Enter type (1-put, 2-get): 1
Enter key and value: 3 30
Enter type (1-put, 2-get): 2
Enter key: 2
Not found
```

2. **Implement a Trie that supports inserting words and searching with wildcards. The . character can match any single character.**

Answer)

```c
// sheikh wasimuddin 63 CSEIOT_CC LAB
#include <stdio.h>
#include <stdlib.h>
struct Trie
{
   struct Trie *child[26];
   int end;
};
struct Trie* createNode()
{
   struct Trie *node = (struct Trie*)malloc(sizeof(struct Trie));
   for (int i = 0; i < 26; i++)
      node->child[i] = NULL;
   node->end = 0;
   return node;
}

void insert(struct Trie *root, char *word)
{
```

```
    struct Trie *cur = root;
    for (int i = 0; word[i]; i++)
    {
        int idx = word[i] - 'a';
        if (!cur->child[idx])
            cur->child[idx] = createNode();
        cur = cur->child[idx];
    }
    cur->end = 1;
}

int search(struct Trie *root, char *word, int pos)
{
    if (!root)
        return 0;

    if (word[pos] == '\0')
        return root->end;

    if (word[pos] == '.')
    {
        for (int i = 0; i < 26; i++)
        {
            if (search(root->child[i], word, pos + 1))
                return 1;
        }
        return 0;
    }
    else
    {
        int idx = word[pos] - 'a';
        return search(root->child[idx], word, pos + 1);
    }
}

int main()
{
    struct Trie *root = createNode();
    int n;
    char word[50];

    printf("Enter number of words: ");
    scanf("%d", &n);

    printf("Enter words:\n");
    for (int i = 0; i < n; i++)
```

```
    {
        scanf("%s", word);
        insert(root, word);
    }

    printf("Enter search pattern: ");
    scanf("%s", word);

    if (search(root, word, 0))
        printf("Word found");
    else
        printf("Word not found");

    return 0;
}
```

main.c

```
// sheikh wasimuddin 63 CSEIOT_CC LAB
#include <stdio.h>
#include <stdlib.h>
struct Trie
{
struct Trie *child[26]; int end;
};
struct Trie* createNode()
{
struct Trie *node = (struct Trie*)malloc(sizeof(struct Trie)); for (int i = 0; i < 26; i++)
```

input

```
Enter number of words: 3
Enter words:
bat
mat
rat
Enter search pattern: ..t
Word found

...Program finished with exit code 0
Press ENTER to exit console.
```

3. **Implement a circular buffer (ring buffer) of size N supporting enqueue and dequeue operations.**

Answer)

```
// sheikh wasimuddin 63 CSEIOT_CC LAB
#include <stdio.h>
int queue[100];
int front = -1, rear = -1, size;

int isFull()
{
   return (front == (rear + 1) % size);
}

int isEmpty()
{
   return (front == -1);
}

void enqueue(int value)
{
   if (isFull())
   {
      printf("Buffer Full\n");
      return;
   }

   if (front == -1)
      front = 0;

   rear = (rear + 1) % size;
   queue[rear] = value;
}

void dequeue()
{
   if (isEmpty())
   {
      printf("Buffer Empty\n");
      return;
   }

   printf("Removed: %d\n", queue[front]);
```

```
    if (front == rear)
    {
        front = rear = -1;
    }
    else
    {
        front = (front + 1) % size;
    }
}

void display()
{
    if (isEmpty())
    {
        printf("Buffer Empty\n");
        return;
    }

    int i = front;
    while (1)
    {
        printf("%d ", queue[i]);
        if (i == rear)
            break;
        i = (i + 1) % size;
    }
    printf("\n");
}

int main()
{
    int n, choice, value;

    printf("Enter buffer size: ");
    scanf("%d", &size);

    printf("Enter number of operations: ");
    scanf("%d", &n);

    while (n--)
    {
        printf("1-Enqueue 2-Dequeue 3-Display: ");
        scanf("%d", &choice);

        if (choice == 1)
        {
```

```
            printf("Enter value: ");
            scanf("%d", &value);
            enqueue(value);
        }
        else if (choice == 2)
        {
            dequeue();
        }
        else
        {
            display();
        }
    }

    return 0;
}
```

main.c

```c
// sheikh wasimuddin 63 CSEIOT_CC LAB
#include <stdio.h>
int queue[100];
int front = -1, rear = -1, size;

int isFull()
{
return (front == (rear + 1) % size);
}

```

input

```
Enter buffer size: 3
Enter number of operations: 5
1-Enqueue 2-Dequeue 3-Display: 1
Enter value: 30
1-Enqueue 2-Dequeue 3-Display: 1
Enter value: 20
1-Enqueue 2-Dequeue 3-Display: 1
Enter value: 10
1-Enqueue 2-Dequeue 3-Display: 2
Removed: 30
1-Enqueue 2-Dequeue 3-Display: 3
20 10
```

4. **Implement an Interval Tree that stores intervals [low, high] and supports efficient queries to find all intervals overlapping with a given interval.**

Answer)

```
// sheikh wasimuddin 63 CSEIOT_CC LAB
#include <stdio.h>
#include <stdlib.h>
struct Node
{
   int low, high, max;
   struct Node *left, *right;
};
struct Node* createNode(int low, int high)
{
   struct Node* node = (struct Node*)malloc(sizeof(struct Node));
   node->low = low;
   node->high = high;
   node->max = high;
   node->left = node->right = NULL;
   return node;
}
int max(int a, int b)
{
   return a > b ? a : b;
}
struct Node* insert(struct Node* root, int low, int high)
{
   if (!root)
      return createNode(low, high);

   if (low < root->low)
      root->left = insert(root->left, low, high);
   else
      root->right = insert(root->right, low, high);

   if (root->max < high)
      root->max = high;

   return root;
}

int overlap(int l1, int h1, int l2, int h2)
{
   return (l1 <= h2 && l2 <= h1);
```

```
}

void searchOverlap(struct Node* root, int low, int high)
{
   if (!root)
      return;

   if (overlap(root->low, root->high, low, high))
      printf("[%d,%d] ", root->low, root->high);

   if (root->left && root->left->max >= low)
      searchOverlap(root->left, low, high);

   searchOverlap(root->right, low, high);
}

int main()
{
   struct Node* root = NULL;
   int n, i, low, high;

   printf("Enter number of intervals: ");
   scanf("%d", &n);

   printf("Enter intervals (low high):\n");
   for (i = 0; i < n; i++)
   {
      scanf("%d %d", &low, &high);
      root = insert(root, low, high);
   }

   printf("Enter query interval: ");
   scanf("%d %d", &low, &high);

   printf("Overlapping intervals: ");
   searchOverlap(root, low, high);

   return 0;
}
```

main.c

```c
// sheikh wasimuddin 63 CSEIOT_CC LAB
#include <stdio.h>
#include <stdlib.h>
struct Node
{
int low, high, max; struct Node *left, *right;
};
struct Node* createNode(int low, int high)
{
struct Node* node = (struct Node*)malloc(sizeof(struct Node)); node->low = low;
```

input

```
Enter number of intervals: 4
Enter intervals (low high):
20 30
10 20
25 35
95 105
Enter query interval: 19 26
Overlapping intervals: [20,30] [10,20] [25,35]
```

5. **Implement a thread-safe queue that supports concurrent enqueue and dequeue operations using mutexes and condition variables.**

Answer)

```c
// sheikh wasimuddin 63 CSEIOT_CC LAB
#include <stdio.h>
#include <stdlib.h>
#include <windows.h>
int queue[100];
int front = 0, rear = 0, count = 0, capacity;
CRITICAL_SECTION lock;
HANDLE not_full, not_empty;
void enqueue(int value)
{
   EnterCriticalSection(&lock);

   while (count == capacity)
   {
      LeaveCriticalSection(&lock);
      WaitForSingleObject(not_full, INFINITE);
      EnterCriticalSection(&lock);
   }
   queue[rear] = value;
   rear = (rear + 1) % capacity;
   count++;
```

```
    SetEvent(not_empty);
    LeaveCriticalSection(&lock);
}
int dequeue()
{
    EnterCriticalSection(&lock);

    while (count == 0)
    {
        LeaveCriticalSection(&lock);
        WaitForSingleObject(not_empty, INFINITE);
        EnterCriticalSection(&lock);
    }

    int value = queue[front];
    front = (front + 1) % capacity;
    count--;

    SetEvent(not_full);
    LeaveCriticalSection(&lock);

    return value;
}

DWORD WINAPI producer(LPVOID arg)
{
    int n = *(int*)arg;
    for (int i = 1; i <= n; i++)
    {
        enqueue(i);
        printf("Produced: %d\n", i);
    }
    return 0;
}

DWORD WINAPI consumer(LPVOID arg)
{
    int n = *(int*)arg;
    for (int i = 1; i <= n; i++)
    {
        int val = dequeue();
        printf("Consumed: %d\n", val);
    }
    return 0;
}
```

```
int main()
{
    int n;

    printf("Enter queue capacity: ");
    scanf("%d", &capacity);

    printf("Enter number of items: ");
    scanf("%d", &n);

    InitializeCriticalSection(&lock);

    not_full = CreateEvent(NULL, FALSE, TRUE, NULL);
    not_empty = CreateEvent(NULL, FALSE, FALSE, NULL);

    HANDLE t1 = CreateThread(NULL, 0, producer, &n, 0, NULL);
    HANDLE t2 = CreateThread(NULL, 0, consumer, &n, 0, NULL);

    WaitForSingleObject(t1, INFINITE);
    WaitForSingleObject(t2,  INFINITE);

    DeleteCriticalSection(&lock);
    CloseHandle(not_full);
    CloseHandle(not_empty);
```

```
#include <stdio.h>
#include <stdlib.h>
#include <windows.h>
int queue[100];
int front = 0, rear = 0, count = 0, capacity;
CRITICAL_SECTION lock;
HANDLE not_full, not_empty;
void enqueue(int value)
{
    EnterCriticalSection(&lock);
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\hp\OneDrive\Desktop\Coding\Advance Ds> cd "c:\Users\hp\OneDrive\Desktop\Coding\Advance Ds\" ; if ($?) { gcc Q5_thread-safe.c -o Q5_thre
Enter queue capacity: 3
Enter number of items: 5
Produced: 1
Produced: 2
Produced: 3
Produced: 4
Consumed: 1
Consumed: 2
Consumed: 3
Consumed: 4
Consumed: 5
Produced: 5
PS C:\Users\hp\OneDrive\Desktop\Coding\Advance Ds>
```

6. **Implement a Skip List that supports efficient insertion, deletion, and search operations.**

Answer)

```
// sheikh wasimuddin 63 CSEIOT_CC LAB
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#define MAX 5
struct Node
{
   int key;
   struct Node* forward[MAX];
};
struct Node* header;
int level = 0;
struct Node* createNode(int key)
{
   struct Node* node = (struct Node*)malloc(sizeof(struct Node));
   node->key = key;
   for (int i = 0; i < MAX; i++)
      node->forward[i] = NULL;
```

```
    return node;
}
int randomLevel()
{
    int lvl = 0;
    while (rand() % 2 && lvl < MAX - 1)
        lvl++;
    return lvl;
}
void insert(int key)
{
    struct Node* update[MAX];
    struct Node* x = header;

    for (int i = level; i >= 0; i--)
    {
        while (x->forward[i] && x->forward[i]->key < key)
            x = x->forward[i];
        update[i] = x;
    }

    x = x->forward[0];

    if (x == NULL || x->key != key)
    {
        int lvl = randomLevel();

        if (lvl > level)
        {
            for (int i = level + 1; i <= lvl; i++)
                update[i] = header;
            level = lvl;
        }

        x = createNode(key);

        for (int i = 0; i <= lvl; i++)
        {
            x->forward[i] = update[i]->forward[i];
            update[i]->forward[i] = x;
        }
    }
}

void deleteKey(int key)
{
```

```
    struct Node* update[MAX];
    struct Node* x = header;

    for (int i = level; i >= 0; i--)
    {
        while (x->forward[i] && x->forward[i]->key < key)
            x = x->forward[i];
        update[i] = x;
    }

    x = x->forward[0];

    if (x && x->key == key)
    {
        for (int i = 0; i <= level; i++)
        {
            if (update[i]->forward[i] != x)
                break;
            update[i]->forward[i] = x->forward[i];
        }

        free(x);

        while (level > 0 && header->forward[level] == NULL)
            level--;
    }
}

int search(int key)
{
    struct Node* x = header;

    for (int i = level; i >= 0; i--)
    {
        while (x->forward[i] && x->forward[i]->key < key)
            x = x->forward[i];
    }

    x = x->forward[0];

    if (x && x->key == key)
        return 1;
    return 0;
}

void display()
```

```
{
    struct Node* x = header->forward[0];
    while (x)
    {
        printf("%d ", x->key);
        x = x->forward[0];
    }
    printf("\n");
}

int main()
{
    int n, choice, key;

    srand(time(0));
    header = createNode(-1);

    printf("Enter number of operations: ");
    scanf("%d", &n);

    while (n--)
    {
        printf("1-Insert 2-Delete 3-Search 4-Display: ");
        scanf("%d", &choice);

        if (choice == 1)
        {
            printf("Enter key: ");
            scanf("%d", &key);
            insert(key);
        }
        else if (choice == 2)
        {
            printf("Enter key: ");
            scanf("%d", &key);
            deleteKey(key);
        }
        else if (choice == 3)
        {
            printf("Enter key: ");
            scanf("%d", &key);
            if (search(key))
                printf("Found\n");
            else
                printf("Not Found\n");
        }
```

```
        else
        {
            display();
        }
    }

    return 0;
}
```

main.c

```
// sheikh wasimuddin 63 CSEIOT_CC LAB
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#define MAX 5
struct Node
{
int key;
struct Node* forward[MAX];
};
```

```
Enter number of operations: 5
1-Insert 2-Delete 3-Search 4-Display: 1
Enter key: 50
1-Insert 2-Delete 3-Search 4-Display: 1
Enter key: 30
1-Insert 2-Delete 3-Search 4-Display: 1
Enter key: 70
1-Insert 2-Delete 3-Search 4-Display: 4
30 50 70
1-Insert 2-Delete 3-Search 4-Display: 3
Enter key: 20
Not Found
```

7. **Implement a B-Tree of minimum degree t that supports insertion, deletion, and search operations.**

Answer)

```
// sheikh wasimuddin 63 CSEIOT_CC LAB
#include <stdio.h>
#include <stdlib.h>
#define T 3
struct Node
{
    int keys[2*T-1];
    struct Node* child[2*T];
    int n;
    int leaf;
};
struct Node* createNode(int leaf)
```

```
{
    struct Node* node = (struct Node*)malloc(sizeof(struct Node));
    node->leaf = leaf;
    node->n = 0;
    for (int i = 0; i < 2*T; i++)
        node->child[i] = NULL;
    return node;
}
void traverse(struct Node* root)
{
    int i;
    for (i = 0; i < root->n; i++)
    {
        if (!root->leaf)
            traverse(root->child[i]);
        printf("%d ", root->keys[i]);
    }
    if (!root->leaf)
        traverse(root->child[i]);
}

struct Node* search(struct Node* root, int k)
{
    int i = 0;
    while (i < root->n && k > root->keys[i])
        i++;

    if (i < root->n && root->keys[i] == k)
        return root;

    if (root->leaf)
        return NULL;

    return search(root->child[i], k);
}

void splitChild(struct Node* x, int i, struct Node* y)
{
    struct Node* z = createNode(y->leaf);
    z->n = T - 1;

    for (int j = 0; j < T - 1; j++)
        z->keys[j] = y->keys[j + T];

    if (!y->leaf)
    {
```

```
        for (int j = 0; j < T; j++)
            z->child[j] = y->child[j + T];
    }

    y->n = T - 1;

    for (int j = x->n; j >= i + 1; j--)
        x->child[j + 1] = x->child[j];

    x->child[i + 1] = z;

    for (int j = x->n - 1; j >= i; j--)
        x->keys[j + 1] = x->keys[j];

    x->keys[i] = y->keys[T - 1];
    x->n++;
}

void insertNonFull(struct Node* x, int k)
{
    int i = x->n - 1;

    if (x->leaf)
    {
        while (i >= 0 && x->keys[i] > k)
        {
            x->keys[i + 1] = x->keys[i];
            i--;
        }
        x->keys[i + 1] = k;
        x->n++;
    }
    else
    {
        while (i >= 0 && x->keys[i] > k)
            i--;

        if (x->child[i + 1]->n == 2*T - 1)
        {
            splitChild(x, i + 1, x->child[i + 1]);
            if (k > x->keys[i + 1])
                i++;
        }
        insertNonFull(x->child[i + 1], k);
    }
}
```

```
struct Node* insert(struct Node* root, int k)
{
   if (root->n == 2*T - 1)
   {
      struct Node* s = createNode(0);
      s->child[0] = root;
      splitChild(s, 0, root);

      int i = 0;
      if (s->keys[0] < k)
         i++;
      insertNonFull(s->child[i],  k);

      return s;
   }
   else
   {
      insertNonFull(root, k);
      return root;
   }
}

int main()
{
   struct Node* root = createNode(1);
   int n, choice, key;

   printf("Enter number of operations: ");
   scanf("%d", &n);

   while (n--)
   {
      printf("1-Insert 2-Search 3-Display: ");
      scanf("%d", &choice);

      if (choice == 1)
      {
         printf("Enter key: ");
         scanf("%d", &key);
         root = insert(root, key);
      }
      else if (choice == 2)
      {
         printf("Enter key: ");
         scanf("%d", &key);
```

```
            if (search(root, key))
                printf("Found\n");
            else
                printf("Not Found\n");
        }
        else
        {
            traverse(root);
            printf("\n");
        }
    }

    return 0;
}
```

main.c

```c
// sheikh wasimuddin 63 CSEIOT_CC LAB
#include <stdio.h>
#include <stdlib.h>
#define T 3
struct Node
{
int keys[2*T-1];
struct Node* child[2*T]; int n;
int leaf;
};
```

```
Enter number of operations: 5
1-Insert 2-Search 3-Display: 1
Enter key: 7
1-Insert 2-Search 3-Display: 1
Enter key: 10
1-Insert 2-Search 3-Display: 1
Enter key: 18
1-Insert 2-Search 3-Display: 3
7 10 18
1-Insert 2-Search 3-Display: 2
Enter key: 9
Not Found
```

8. Implement a Union-Find (Disjoint Set) data structure with path compression and union by rank.

Answer)

```
// sheikh wasimuddin 63 CSEIOT_CC LAB
#include <stdio.h>
int parent[100], rankArr[100];
void makeSet(int n)
{
   for(int i=0;i<n;i++)
   {
      parent[i]=i;
      rankArr[i]=0;
   }
}
int find(int x)
{
   if(parent[x]!=x)
      parent[x]=find(parent[x]);
   return parent[x];
}
void unionSet(int x,int y)
{
   int rootX=find(x);
   int rootY=find(y);
   if(rootX!=rootY)
   {
      if(rankArr[rootX]<rankArr[rootY])
         parent[rootX]=rootY;
      else if(rankArr[rootX]>rankArr[rootY])
         parent[rootY]=rootX;
      else
      {
         parent[rootY]=rootX;
         rankArr[rootX]++;
      }
   }
}

int main()
{
   int n,m;

   printf("Enter number of elements: ");
   scanf("%d",&n);
```

```
    makeSet(n);

    printf("Enter number of unions: ");
    scanf("%d",&m);

    for(int i=0;i<m;i++)
    {
        int x,y;
        printf("Enter pair %d: ",i+1);
        scanf("%d %d",&x,&y);
        unionSet(x,y);
    }

    printf("Set representatives:\n");
    for(int i=0;i<n;i++)
        printf("%d ",find(i));

    return 0;
}
```

main.c

```
// sheikh wasimuddin 63 CSEIOT_CC LAB
#include <stdio.h>
int parent[100],rankArr[100];
void makeSet(int n)
{
for(int i=0;i<n;i++)
{
parent[i]=i; rankArr[i]=0;
}
}
```

```
Enter number of elements: 5
Enter number of unions: 3
Enter pair 1: 0 1
Enter pair 2: 1 2
Enter pair 3: 2 3
Set representatives:
0 0 0 0 4
```

9. **Given a weighted directed graph, implement Dijkstra's Algorithm using adjacency lists to find the shortest distance from a source vertex to all others.**

Answer)

```
// sheikh wasimuddin 63 CSEIOT_CC LAB
#include <stdio.h>
#define MAX 100
#define INF 9999
int n, e;
int adj[MAX][MAX];
void dijkstra(int src)
{
   int dist[MAX], visited[MAX];

   for(int i=0;i<n;i++)
   {
      dist[i]=INF;
      visited[i]=0;
   }
   dist[src]=0;
   for(int i=0;i<n-1;i++)
   {
      int min=INF, u;
      for(int j=0;j<n;j++)
      {
         if(!visited[j] && dist[j]<min)
         {
            min=dist[j];
            u=j;
         }
      }
      visited[u]=1;
      for(int v=0;v<n;v++)
      {
         if(!visited[v] && adj[u][v]!=INF && dist[u]+adj[u][v]<dist[v])
            dist[v]=dist[u]+adj[u][v];
      }
   }

   printf("Shortest distances from source:\n");
   for(int i=0;i<n;i++)
      printf("To %d = %d\n",i,dist[i]);
}
```

```
int main()
{
    printf("Enter number of vertices: ");
    scanf("%d",&n);

    printf("Enter number of edges: ");
    scanf("%d",&e);

    for(int i=0;i<n;i++)
        for(int j=0;j<n;j++)
            adj[i][j]=INF;

    printf("Enter edges (u v weight):\n");
    for(int i=0;i<e;i++)
    {
        int u,v,w;
        scanf("%d %d %d",&u,&v,&w);
        adj[u][v]=w;
    }

    int src;
    printf("Enter source vertex: ");
    scanf("%d",&src);

    dijkstra(src);

    return 0;
}
```

main.c

```c
// sheikh wasimuddin 63 CSEIOT_CC LAB
#include <stdio.h>
#define MAX 100
#define INF 9999
int n, e;
int adj[MAX][MAX];
void dijkstra(int src)
{
int dist[MAX], visited[MAX];
```

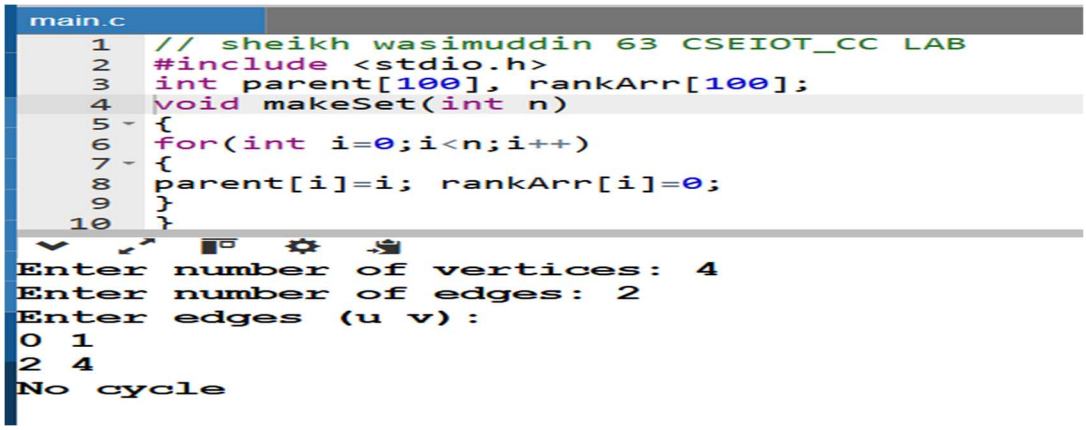
```
Enter number of vertices: 4
Enter number of edges: 5
Enter edges (u v weight):
0 2 1
2 0 4
1 2 1
3 7 1
2 4 3
Enter source vertex: 1
Shortest distances from source:
To 0 = 5
To 1 = 0
To 2 = 1
To 3 = 9999
```

10. Detect whether a given undirected graph contains a cycle using Union-Find.

Answer)

```c
// sheikh wasimuddin 63 CSEIOT_CC LAB
#include <stdio.h>
int parent[100], rankArr[100];
void makeSet(int n)
{
   for(int i=0;i<n;i++)
   {
      parent[i]=i;
      rankArr[i]=0;
   }
}
int find(int x)
{
   if(parent[x]!=x)
      parent[x]=find(parent[x]);
   return parent[x];
}
int unionSet(int x,int y)
```

```
{
    int rootX=find(x);
    int rootY=find(y);

    if(rootX==rootY)
        return 1;
    if(rankArr[rootX]<rankArr[rootY])
        parent[rootX]=rootY;
    else if(rankArr[rootX]>rankArr[rootY])
        parent[rootY]=rootX;
    else
    {
        parent[rootY]=rootX;
        rankArr[rootX]++;
    }
    return 0;
}
int main()
{
    int n,e;
    printf("Enter number of vertices: ");
    scanf("%d",&n);

    printf("Enter number of edges: ");
    scanf("%d",&e);

    makeSet(n);

    int cycle=0;

    printf("Enter edges (u v):\n");
    for(int i=0;i<e;i++)
    {
        int u,v;
        scanf("%d %d",&u,&v);

        if(unionSet(u,v))
            cycle=1;
    }

    if(cycle)
        printf("Cycle detected\n");
    else
        printf("No cycle\n");

    return 0; }
```



11. Implement a memory-efficient representation of a sparse matrix.

Answer)

```
// sheikh wasimuddin 63 CSEIOT_CC LAB
#include <stdio.h>
int main()
{
   int rows, cols;
   printf("Enter rows and columns: ");
   scanf("%d %d",&rows,&cols);
   int matrix[100][100];
   printf("Enter matrix elements:\n");
   for(int i=0;i<rows;i++)
      for(int j=0;j<cols;j++)
         scanf("%d",&matrix[i][j]);
   int sparse[100][3];
   int k=0;
   for(int i=0;i<rows;i++)
   {
      for(int j=0;j<cols;j++)
      {
```

```
            if(matrix[i][j]!=0)
            {
                sparse[k][0]=i;
                sparse[k][1]=j;
                sparse[k][2]=matrix[i][j];
                k++;
            }
        }
    }
    printf("Sparse Matrix (row col value):\n");
    for(int i=0;i<k;i++)
        printf("%d %d %d\n",sparse[i][0],sparse[i][1],sparse[i][2]);
    return 0;
}
```

main.c

```c
// sheikh wasimuddin 63 CSEIOT_CC LAB
#include <stdio.h>
int main()
{
int rows, cols;
printf("Enter rows and columns: "); scanf("%d %d",&rows,&cols);
int matrix[100][100]; printf("Enter matrix elements:\n"); for(int i=0;i<rows;i++)
for(int j=0;j<cols;j++) scanf("%d",&matrix[i][j]);
int sparse[100][3]; int k=0;
for(int i=0;i<rows;i++)
```

input

```
Enter rows and columns: 3 3
Enter matrix elements:
1 0 1
0 1 0
1 0 1
Sparse Matrix (row col value):
0 0 1
0 2 1
1 1 1
2 0 1
2 2 1
```

12. Implement a Bloom Filter for strings with multiple hash functions.

Answer)

```
// sheikh wasimuddin 63 CSEIOT_CC LAB
#include <stdio.h>
#include <string.h>
#define SIZE 100
int bitArray[SIZE];
int hash1(char *str)
```

```
{
    int hash=0;

    for(int i=0;str[i];i++)
        hash=(hash+str[i])%SIZE;
    return hash;
}
int hash2(char *str)
{
    int hash=0;
    for(int i=0;str[i];i++)
        hash=(hash*31+str[i])%SIZE;
    return hash;
}
void insert(char *str)
{
    int h1=hash1(str);
    int h2=hash2(str);

    bitArray[h1]=1;
    bitArray[h2]=1;
}
int check(char *str)
{
    int h1=hash1(str);
    int h2=hash2(str);

    if(bitArray[h1] && bitArray[h2])
        return 1;
    return 0;
}
int main()
{
    int n;
    char str[50];

    printf("Enter number of strings to insert: ");
    scanf("%d",&n);

    for(int i=0;i<n;i++)
    {
        printf("Enter string %d: ",i+1);
        scanf("%s",str);
        insert(str);
    }

    int q;
    printf("Enter number of queries: ");
```

```
    scanf("%d",&q);


    for(int i=0;i<q;i++)
    {
        printf("Enter string to check: ");
        scanf("%s",str);

        if(check(str))
            printf("Possibly present\n");
        else
            printf("Definitely not present\n");
    }

    return 0;
}
```

main.c

```c
// sheikh wasimuddin 63 CSEIOT_CC LAB
#include <stdio.h>
#include <string.h>
#define SIZE 100
int bitArray[SIZE];
int hash1(char *str)
{
int hash=0;

for(int i=0;str[i];i++) hash=(hash+str[i])%SI
```

```
Enter number of strings to insert: 4
Enter string 1: cat
Enter string 2: bat
Enter string 3: mat
Enter string 4: rat
Enter number of queries: 4
Enter string to check: bat
Possibly present
Enter string to check: pat
Definitely not present
Enter string to check: rat
Possibly present
Enter string to check: net
Definitely not present
```

13. Implement a Segment Tree supporting: 1) Range sum queries 2) Range add updates

Answer)

```
// sheikh wasimuddin 63 CSEIOT_CC LAB
#include <stdio.h>
#define MAX 100
```

```
int arr[MAX], tree[4*MAX], lazy[4*MAX];
void build(int node,int start,int end)
{
   if(start==end)
      tree[node]=arr[start];

   else
   {
      int mid=(start+end)/2;
      build(2*node,start,mid);
      build(2*node+1,mid+1,end);
      tree[node]=tree[2*node]+tree[2*node+1];
   }
}
void update(int node,int start,int end,int l,int r,int val)
{
   if(lazy[node]!=0)
   {
      tree[node]+=(end-start+1)*lazy[node];
      if(start!=end)
      {
         lazy[2*node]+=lazy[node];
         lazy[2*node+1]+=lazy[node];
      }
      lazy[node]=0;
   }
   if(start>r || end<l)
      return;
   if(start>=l && end<=r)
   {
      tree[node]+=(end-start+1)*val;
      if(start!=end)
      {
         lazy[2*node]+=val;
         lazy[2*node+1]+=val;
      }
      return;
   }
   int mid=(start+end)/2;
   update(2*node,start,mid,l,r,val);
   update(2*node+1,mid+1,end,l,r,val);

   tree[node]=tree[2*node]+tree[2*node+1];
}
int query(int node,int start,int end,int l,int r)
{
   if(start>r || end<l)
      return 0;
```

```
    if(lazy[node]!=0)
    {
        tree[node]+=(end-start+1)*lazy[node];

        if(start!=end)
        {
            lazy[2*node]+=lazy[node];
            lazy[2*node+1]+=lazy[node];
        }
        lazy[node]=0;
    }
    if(start>=l && end<=r)
        return tree[node];

    int mid=(start+end)/2;
    return query(2*node,start,mid,l,r) + query(2*node+1,mid+1,end,l,r);
}
int main()
{
    int n;
    printf("Enter number of elements: ");
    scanf("%d",&n);
    printf("Enter elements:\n");
    for(int i=0;i<n;i++)
        scanf("%d",&arr[i]);
    build(1,0,n-1);
    int q;
    printf("Enter number of operations: ");
    scanf("%d",&q);
    for(int i=0;i<q;i++)
    {
        int type;
        printf("1: Update  2: Query\nEnter type: ");
        scanf("%d",&type);

        if(type==1)
        {
            int l,r,val;
            printf("Enter l r value: ");
            scanf("%d %d %d",&l,&r,&val);
            update(1,0,n-1,l,r,val);
        }
        else
        {
            int l,r;
            printf("Enter l r: ");
            scanf("%d %d",&l,&r);
```

```
        printf("Sum = %d\n",query(1,0,n-1,l,r));
      }
    }


    return 0;
  }
```

```
main.c
// sheikh wasimuddin 63 CSEIOT_CC LAB
#include <stdio.h>
#define MAX 100
int arr[MAX], tree[4*MAX], lazy[4*MAX];
void build(int node,int start,int end)
{
if(start==end) tree[node]=arr[start];

else
```

```
Enter number of elements: 5
Enter elements:
9 8 7 6 5
Enter number of operations: 5
1: Update 2: Query
Enter type: 1
Enter l r value: 1 2 3 4 5
1: Update 2: Query
Enter type: Enter l r: 2
Sum = 0
1: Update 2: Query
Enter type: 2
Enter l r: 0 4
Sum = 41
1: Update 2: Query
Enter type: 1
Enter l r value: 2 3 5
```

14. Given an array, count the number of inversions (i, j) where i<t; j and arr[i] > arr[j] using Binary Indexed Tree (Fenwick Tree).

Answer)

```
// sheikh wasimuddin 63 CSEIOT_CC LAB
#include <stdio.h>
#define MAX 100
int BIT[MAX], n;
void update(int i,int val)
{
  while(i<=n)
  {
    BIT[i]+=val;
    i+=i&(-i);
  }
}
```

```
int query(int i)
{
    int sum=0;
    while(i>0)
    {
        sum+=BIT[i];
        i-=i&(-i);
    }
    return sum;
}
int main()
{
    int arr[MAX], temp[MAX];

    printf("Enter number of elements: ");
    scanf("%d",&n);

    printf("Enter elements:\n");
    for(int i=1;i<=n;i++)
    {
        scanf("%d",&arr[i]);
        temp[i]=arr[i];
    }
    for(int i=1;i<=n;i++)
    {
        for(int j=i+1;j<=n;j++)
        {
            if(temp[i]>temp[j])
            {
                int t=temp[i];
                temp[i]=temp[j];
                temp[j]=t;
            }
        }
    }
    for(int i=1;i<=n;i++)
    {
        for(int j=1;j<=n;j++)
        {
            if(arr[i]==temp[j])
            {
                arr[i]=j;
                break;
            }
        }
    }
    int inv=0;
    for(int i=n;i>=1;i--)
```

```
    {
        inv+=query(arr[i]-1);
        update(arr[i],1);

    }
    printf("Number of inversions = %d\n",inv);
    return 0;
}
```

main.c

```
// sheikh wasimuddin 63 CSEIOT_CC LAB
#include <stdio.h>
#define MAX 100
int BIT[MAX], n;
void update(int i,int val)
{
while(i<=n)
{
BIT[i]+=val; i+=i&(-i);
}
```

```
Enter number of elements: 6
Enter elements:
9 8 7 6 5 4
Number of inversions = 15
```

15. Design a data structure Fibonacci Heap in C that supports efficient priority queue operations.

Answer)

```
// sheikh wasimuddin 63 CSEIOT_CC LAB
#include <stdio.h>
#include <stdlib.h>
typedef struct node
{
    int key, degree;
    struct node *parent, *child, *left, *right;
}node;
node *min=NULL;
node* createNode(int key)
{
    node* newNode=(node*)malloc(sizeof(node));
    newNode->key=key;
    newNode->degree=0;
    newNode->parent=newNode->child=NULL;
    newNode->left=newNode->right=newNode;
    return newNode;
```

```
}
void insert(int key)
{
   node* x=createNode(key);
   if(min==NULL)
      min=x;
   else
   {
      x->left=min;
      x->right=min->right;
      min->right->left=x;
      min->right=x;

      if(x->key<min->key)
         min=x;
   }
}
node* getMin()
{
   return min;
}
void link(node* y,node* x)
{
   y->left->right=y->right;
   y->right->left=y->left;
   y->parent=x;
   if(x->child==NULL)
   {
      x->child=y;
      y->left=y->right=y;
   }
   else
   {
      y->left=x->child;
      y->right=x->child->right;
      x->child->right->left=y;
      x->child->right=y;
   }

   x->degree++;
}
void consolidate()
{
   node* A[100]={0};
   node* w=min;
   if(w==NULL) return;
```

```
    node* start=w;
    do
    {
        node* x=w;
        int d=x->degree;
        while(A[d]!=NULL)
        {
            node* y=A[d];
            if(x->key>y->key)
            {
                node* temp=x;
                x=y;
                y=temp;
            }
            link(y,x);
            A[d]=NULL;
            d++;
        }
        A[d]=x;
        w=w->right;
    }while(w!=start);
    min=NULL;
    for(int i=0;i<100;i++)
    {
        if(A[i]!=NULL)
        {
            if(min==NULL)
                min=A[i];
            else if(A[i]->key<min->key)
                min=A[i];
        }
    }
}
void extractMin()
{
    node* z=min;
    if(z!=NULL)
    {
        if(z->child!=NULL)
        {
            node* c=z->child;
            node* start=c;
            do
            {
                node* next=c->right;
                c->left=min;
```

```
            c->right=min->right;
            min->right->left=c;
            min->right=c;
            c->parent=NULL;
            c=next;
         }while(c!=start);
      }
      z->left->right=z->right;
      z->right->left=z->left;
      if(z==z->right)
         min=NULL;
      else
      {
         min=z->right;
         consolidate();
      }
   }
}
int main()
{
   int choice, val;
   while(1)
   {
      printf("\n1.Insert 2.GetMin 3.ExtractMin 4.Exit\n");
      printf("Enter choice: ");
      scanf("%d",&choice);
      if(choice==1)
      {
         printf("Enter value: ");
         scanf("%d",&val);
         insert(val);
      }
      else if(choice==2)
      {
         if(min)
            printf("Minimum = %d\n",getMin()->key);
         else
            printf("Heap is empty\n");
      }
      else if(choice==3)
      {
         if(min)
         {
            printf("Extracted Min = %d\n",min->key);
            extractMin();
         }
```

```
            else
                printf("Heap is empty\n");
        }
        else
            break;
    }
    return 0;
}
```

main.c

```c
// sheikh wasimuddin 63 CSEIOT_CC LAB
#include <stdio.h>
#include <stdlib.h>
typedef struct node
{
int key, degree;
struct node *parent, *child, *left, *right;
}node;
node *min=NULL;
node* createNode(int key)
```

```
1.Insert 2.GetMin 3.ExtractMin 4.Exit
Enter choice: 1
Enter value: 77

1.Insert 2.GetMin 3.ExtractMin 4.Exit
Enter choice: 1
Enter value: 99

1.Insert 2.GetMin 3.ExtractMin 4.Exit
Enter choice: 1
Enter value: 268

1.Insert 2.GetMin 3.ExtractMin 4.Exit
Enter choice: 3
Extracted Min = 77

1.Insert 2.GetMin 3.ExtractMin 4.Exit
Enter choice: 2
Minimum = 99
```